

ESP32 EMBEDDED TUTORIAL

ESP32 与 MPU6050 姿态传感器入门教程

从硬件接线、I2C 通信、寄存器读取，到加速度、角速度和姿态角的完整实践。所有示例只使用 Arduino 内置 Wire 库，不依赖第三方库。

适用平台：Arduino IDE / ESP32

对应源码：mpu.ino

更新日期：2026-09

目录

1. 认识 MPU6050
2. 接线与 I2C 地址
3. Arduino IDE 准备
4. 首次运行与观察
5. 完整示例代码
6. 代码要点解析
7. 数据换算与姿态角
8. 常见问题排查
9. 后续学习路线

1. 认识 MPU6050

MPU6050 是一种常见的 6 轴运动传感器。它内部同时包含：

- 3 轴加速度计，用于测量 x、y、z 方向的加速度，单位通常换算为 g。
- 3 轴陀螺仪，用于测量绕 x、y、z 轴旋转的角速度，单位通常换算为 deg/s。

它本身没有磁力计，所以单靠 MPU6050 无法可靠得到水平方向上的绝对航向 yaw。

本教程的目标：完成一次从 ESP32 读取 MPU6050 原始数据、做陀螺仪零偏校准、计算 pitch/roll，并通过串口输出结果的完整流程。

2. 接线与 I2C 地址

以常见的 ESP32 DevKit 开发板为例，默认使用 GPIO 21 作为 SDA、GPIO 22 作为 SCL。

ESP32	MPU6050 模块	说明
3V3	VCC	优先使用 3.3V 电源
GND	GND	公共地
GPIO 21	SDA	I2C 数据线
GPIO 22	SCL	I2C 时钟线
GND	AD0	地址选择引脚，接 GND 时为 0x68

I2C 地址由 AD0 引脚决定：

AD0 连接	I2C 地址	代码中的设置
GND	0x68	#define MPU6050_ADDR 0x68
3V3	0x69	#define MPU6050_ADDR 0x69

接线注意：建议使用 3.3V。如果模块说明明确支持 5V，也不要只把 SDA/SCL 直接接 5V，避免电平不匹配。

3. Arduino IDE 准备

- 在 Arduino IDE 中安装 ESP32 开发板支持，搜索开发板核心 `esp32`。
- 选择正确的板型，例如 `ESP32 Dev Module`。
- 选择正确的串口，然后上传 `mpu.ino`。
- 打开串口监视器，波特率设置为 `115200`。

该示例只包含 `Wire.h`，这是 Arduino/ESP32 核心自带的库，不需要从库管理器中安装 MPU6050 库。

如果使用的不是 ESP32 DevKit，而是 ESP32-S3、ESP32-C3 或其他板型，请先查看该板的 I2C 引脚表，并修改代码开头的 `MPU6050_SDA` 和 `MPU6050_SCL`。

4. 首次运行与观察

上电或复位后，程序会先读取 `WHO_AM_I` 寄存器确认设备存在，然后唤醒 MPU6050 并配置测量范围。接着它会采集 500 个静止样本计算陀螺仪零偏。

校准期间必须保持传感器静止。如果校准时传感器在转动，静止后的角速度会带有反向漂移。

正常启动时串口会显示：

```
MPU6050 demo
Calibrating gyro. Keep the sensor still...
Calibration done.
ax/ay/az in g | gyro in deg/s | pitch/roll in degrees
```

随后每 50ms 输出一行数据：

```
ax= 0.01 ay=-0.02 az= 1.00 | gx= 0.12 gy= 0.08 gz=-0.05 | pitch= -0.6 roll= 0.8
```

将模块平放时，`az` 应该接近 `1.00 g`。分别绕三个轴缓慢转动，可以观察到对应的 `gx`、`gy`、`gz` 发生变化。

5. 完整示例代码

以下是 `mpu.ino` 的完整内容。该程序实现寄存器读写、陀螺仪校准、传感器读取和姿态角计算。

```
/*
 * Minimal MPU6050 example for ESP32 using only Wire.
 *
 * Default wiring:
 *   ESP32 3V3 -> VCC
 *   ESP32 GND -> GND
 *   GPIO 21  -> SDA
 *   GPIO 22  -> SCL
 *   GND      -> AD0 (sets I2C address to 0x68)
 */

#include <Wire.h>

#define MPU6050_ADDR    0x68
#define MPU6050_SDA     21
#define MPU6050_SCL     22

#define REG_PWR_MGMT_1  0x6B
#define REG_SMPLRT_DIV  0x19
#define REG_CONFIG      0x1A
#define REG_GYRO_CONFIG  0x1B
#define REG_ACCEL_CONFIG 0x1C
#define REG_ACCEL_XOUT_H 0x3B
#define REG_WHO_AM_I    0x75

constexpr float ACCEL_SCALE = 16384.0f; // +/-2g
constexpr float GYRO_SCALE = 131.0f;    // +/-250 deg/s
constexpr int CALIBRATION_SAMPLES = 500;
constexpr unsigned long PRINT_INTERVAL_MS = 50;

struct SensorData {
    int16_t accelX;
    int16_t accelY;
    int16_t accelZ;
    int16_t temp;
    int16_t gyroX;
    int16_t gyroY;
    int16_t gyroZ;
};

float gyroBiasX = 0.0f;
float gyroBiasY = 0.0f;
float gyroBiasZ = 0.0f;

bool writeRegister(uint8_t reg, uint8_t value) {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(reg);
    Wire.write(value);
    return Wire.endTransmission() == 0;
}

bool readBlock(uint8_t reg, uint8_t *data, size_t length) {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(reg);

    if (Wire.endTransmission(false) != 0) {
```

```

    return false;
}

size_t received = Wire.requestFrom(static_cast<int>(MPU6050_ADDR), length);
if (received < length) {
    while (Wire.available()) {
        Wire.read();
    }
    return false;
}

for (size_t i = 0; i < length; ++i) {
    data[i] = static_cast<uint8_t>(Wire.read());
}
return true;
}

uint8_t readRegister(uint8_t reg) {
    uint8_t value = 0;
    return readBlock(reg, &value, 1) ? value : 0xFF;
}

int16_t combineBytes(uint8_t high, uint8_t low) {
    return static_cast<int16_t>((static_cast<uint16_t>(high) << 8) | low);
}

bool readSensor(SensorData &data) {
    uint8_t raw[14] = {0};
    if (!readBlock(REG_ACCEL_XOUT_H, raw, sizeof(raw))) {
        return false;
    }

    data.accelX = combineBytes(raw[0], raw[1]);
    data.accelY = combineBytes(raw[2], raw[3]);
    data.accelZ = combineBytes(raw[4], raw[5]);
    data.temp = combineBytes(raw[6], raw[7]);
    data.gyroX = combineBytes(raw[8], raw[9]);
    data.gyroY = combineBytes(raw[10], raw[11]);
    data.gyroZ = combineBytes(raw[12], raw[13]);
    return true;
}

void calibrateGyro() {
    SensorData data;
    float sumX = 0.0f;
    float sumY = 0.0f;
    float sumZ = 0.0f;
    int validSamples = 0;

    Serial.println("Calibrating gyro. Keep the sensor still...");
    for (int i = 0; i < CALIBRATION_SAMPLES; ++i) {
        if (readSensor(data)) {
            sumX += data.gyroX;
            sumY += data.gyroY;
            sumZ += data.gyroZ;
            ++validSamples;
        }
        delay(1);
    }

    if (validSamples > 0) {
        gyroBiasX = sumX / validSamples;
        gyroBiasY = sumY / validSamples;
    }
}

```

```

    gyroBiasZ = sumZ / validSamples;
}
}

void setup() {
    Serial.begin(115200);
    Wire.begin(MPU6050_SDA, MPU6050_SCL);
    Wire.setClock(400000);
    delay(100);

    Serial.println("MPU6050 demo");

    uint8_t whoAmI = readRegister(REG_WHO_AM_I);
    if ((whoAmI & 0x7E) != 0x68) {
        Serial.printf("MPU6050 not found on I2C address 0x%02X. WHO_AM_I=0x%02X\n",
            MPU6050_ADDR, whoAmI);
        while (true) {
            delay(1000);
        }
    }

    if (!writeRegister(REG_PWR_MGMT_1, 0x00) ||
        !writeRegister(REG_SMPLRT_DIV, 0x00) ||
        !writeRegister(REG_CONFIG, 0x03) ||
        !writeRegister(REG_GYRO_CONFIG, 0x00) ||
        !writeRegister(REG_ACCEL_CONFIG, 0x00)) {
        Serial.println("Failed to configure MPU6050.");
        while (true) {
            delay(1000);
        }
    }

    delay(100);
    calibrateGyro();
    Serial.println("Calibration done.");
    Serial.println("ax/ay/az in g | gyro in deg/s | pitch/roll in degrees");
}

void loop() {
    static unsigned long lastPrint = 0;
    static unsigned long lastErrorPrint = 0;
    SensorData data;

    if (!readSensor(data)) {
        unsigned long now = millis();
        if (now - lastErrorPrint >= 1000) {
            Serial.println("I2C read error. Check SDA/SCL and power wiring.");
            lastErrorPrint = now;
        }
        return;
    }

    float accelX = data.accelX / ACCEL_SCALE;
    float accelY = data.accelY / ACCEL_SCALE;
    float accelZ = data.accelZ / ACCEL_SCALE;

    float gyroX = (data.gyroX - gyroBiasX) / GYRO_SCALE;
    float gyroY = (data.gyroY - gyroBiasY) / GYRO_SCALE;
    float gyroZ = (data.gyroZ - gyroBiasZ) / GYRO_SCALE;

    float pitch = atan2(-accelX, sqrt(accelY * accelY + accelZ * accelZ)) * 180.0f / PI;
    float roll = atan2(accelY, accelZ) * 180.0f / PI;

```

```
unsigned long now = millis();
if (now - lastPrint >= PRINT_INTERVAL_MS) {
  Serial.printf(
    "ax=%7.2f ay=%7.2f az=%7.2f | "
    "gx=%7.2f gy=%7.2f gz=%7.2f | "
    "pitch=%7.1f roll=%7.1f\n",
    accelX, accelY, accelZ,
    gyroX, gyroY, gyroZ,
    pitch, roll);
  lastPrint = now;
}
}
```

6. 代码要点解析

6.1 初始化 MPU6050

示例中使用的关键寄存器如下：

寄存器	地址	作用
PWR_MGMT_1	0x6B	写入 0x00 唤醒设备
SMPLRT_DIV	0x19	采样率分频，0 表示不额外分频
CONFIG	0x1A	数字低通滤波器配置
GYRO_CONFIG	0x1B	陀螺仪量程配置
ACCEL_CONFIG	0x1C	加速度计量程配置
ACCEL_XOUT_H	0x3B	加速度数据寄存器起始地址
WHO_AM_I	0x75	设备识别寄存器

6.2 I2C 读取

程序先写寄存器地址，再从 0x3B 开始连续读取 14 个字节。寄存器顺序为：

- 加速度 X、Y、Z，各 2 个字节，共 6 字节。
- 温度 2 个字节。
- 陀螺仪 X、Y、Z，各 2 个字节，共 6 字节。

每个数据都是高位在前、低位在后的有符号 16 位整数。

6.3 陀螺仪零偏校准

传感器静止时，陀螺仪理论上输出 0，但实际通常存在少量零偏。程序采集 500 个样本并计算平均值，之后每次读取都减去该零偏。这只能消除固定零偏，长时间积分仍然会漂移。

7. 数据换算与姿态角

本示例配置加速度计量程为 $\pm 2g$ ，陀螺仪量程为 $\pm 250 \text{ deg/s}$ ，因此换算公式为：

```
accel_g = accel_raw / 16384 gyro_dps = (gyro_raw - gyro_bias) / 131 temperature_c = temp_raw /  
340 + 36.53
```

从加速度计得到的 pitch 和 roll 使用以下公式：

```
pitch = atan2(-ax, sqrt(ay^2 + az^2)) * 180 / pi roll = atan2(ay, az) * 180 / pi
```

当模块静止平放时，az 约为 1g；倾斜模块时，pitch 和 roll 会跟着变化。

姿态角限制：这里的 pitch/roll 只来自加速度计，适合缓慢倾斜测量。运动时的线性加速度会让结果抖动。陀螺仪短期很灵敏，但积分后角度会随时间漂移。yaw 无法仅由 MPU6050 准确得到。

8. 常见问题排查

现象	可能原因与处理方法
提示 MPU6050 not found	检查 SDA、SCL、VCC、GND；确认 AD0 地址；检查 GPIO 21/22 是否被其他外设占用。
串口乱码	串口监视器波特率必须是 115200。
数值一直为 0	MPU6050 可能未唤醒，或 I2C 连接不稳定；重新上电并观察启动信息。
静止时陀螺仪仍在漂移	校准过程中传感器没有保持静止，请重新复位并保持静止。
pitch/roll 在运动时抖动	这是纯加速度计姿态角的正常现象，使用互补滤波或姿态解算库可以改善。
出现 I2C read error	检查杜邦线是否松动，避免线材过长；可临时把 I2C 时钟从 400kHz 降为 100kHz。

9. 后续学习路线

- 用互补滤波融合加速度计和陀螺仪，获得更稳定的 pitch/roll。
- 学习卡尔曼滤波，理解过程噪声和观测噪声。
- 使用 MPU6050_light、Adafruit MPU6050 等库快速获得姿态角。
- 把 pitch/roll 显示到 OLED，或用 WiFi/BLE 发送到电脑和手机。
- 添加磁力计，例如 QMC5883L，实现更完整的 9 轴姿态估计。

先理解原始寄存器读取，再使用第三方姿态库，会更容易判断哪些数值可信，也能更快定位硬件和通信问题。